

In the following $A, B, \dots$ are propositional variables, $a, b, \dots$ constant symbols, $f, g, \dots$ function symbols, $X, Y \dots$ variables, $p, q, \dots$ predicate symbols and $\phi, \psi, \dots$ formulas (unless differently specified).

1. (4 points) Consider the language of propositional logic. Use natural deduction to prove that the following holds, or find a counter-example to show that it does not hold

- $\vdash (\neg\phi \rightarrow \neg\psi) \rightarrow (\psi \rightarrow \phi)$
- $\vdash (\phi \vee \psi) \rightarrow (\neg(\neg\phi \wedge \neg\psi))$

Assume that you can use the rule for $\vee$ introduction

$$\frac{\phi}{\phi \vee \psi}$$

2. (4 points) Transform the following propositional logic formula into an equivalent formula in Conjunctive Normal Form

- $\neg A \wedge (\neg B \wedge ((C \wedge D) \rightarrow A))$

3. (4 points) Prove that that $\phi \models \psi$ (ψ is a logical consequence of ϕ) or that $\phi \not\models \psi$ for the following formulas:

- $\phi : \neg A \vee \neg B$ and $\psi : (A \wedge \neg B) \rightarrow (\neg B \vee C)$
- $\phi : A \vee C$ and $\psi : (A \rightarrow C) \rightarrow (C \rightarrow A)$

4. Anna and Max carpool to work. On any day, either Anna drives Barbara or Barbara drives Anna. In the former case, Anna is the driver and Barbara is the passenger; in the latter case Barbara is the driver and Anna is the passenger. The roles of driver and passenger are exchanged every two days. Formalize the problem using propositional logic.

5. (5 points) Consider the following definitions for a graph G :

- Adjacent node: a node x is adjacent to node y iff there exists an edge between x and y .
- Path: a path in a graph is a sequence of edges which joins a sequence of nodes.
- Distance: the distance of two nodes x and y is the number of edges needed to obtain a path from x to y .
- Connected component: A connected component in G is a subgraph of G in which any two vertices are connected to each other by paths, and which is connected to no additional vertices in the supergraph G .
- Alternate graph coloring problem: given a non-oriented graph G such that
 - * for any pairs of node x and y in G there exists at most one path from x to y ,associate a color to each of its nodes in such a way that adjacent nodes have different colors, while nodes which are distant 2 have the same color.

Provide a FOL language and a set of axioms that formalizes the alternate graph coloring problem of a graph with k connected components, minimizing the number of colors used. Note: you do not need to formalize *, that is, you can assume that G is a set of trees.

6. (5 points) A binary tree is either empty or it is composed of a root element and two successors, which are binary trees themselves. In Prolog we represent the empty tree by the atom `nil` and the non-empty tree by the term `t(X,L,R)` where X denotes the root node and L and R denote the left and right subtree, respectively. A leaf is a node with no successors. Two nodes are siblings if they have the same parent. For example, the tree in the Figure is represented by the term `t(a,t(b,nil,nil),t(c,nil,nil))` and b and c are leaves.

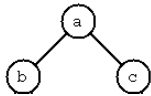


Figure 1: A tree

Write a Prolog program which defines a predicate `count_leaves_without_sib(T,N)` which counts the number of leafs which have no siblings

```
% count_leaves_with_sib(T,N) :- the binary tree T has N leafs which  
% no siblings.
```

7. (4 points) Provide a Prolog program P and a goal G such that the evaluation of G in P successfully terminates, while the evaluation of G in a program P' , obtained from P by changing the order of atoms in the body of a clause, does not terminate.
8. (3 points) Describe shortly the difference between Prolog and Minizinc.

Es. 1

1) $(\neg \Phi \rightarrow \neg \Psi) \rightarrow (\Psi \rightarrow \Phi) \stackrel{?}{=} \perp$ so always true

$$(\neg \Phi \rightarrow \neg \Psi) \equiv \top \quad (\top \rightarrow \perp) \equiv \perp$$

$$(\Psi \rightarrow \Phi) \equiv \perp \quad \Phi \equiv \perp, \Psi \equiv \top$$

2) $(\Phi \vee \Psi) \rightarrow (\neg(\neg \Phi \wedge \neg \Psi)) \stackrel{?}{=} \perp$ so always true

$$\Phi \vee \Psi \equiv \top$$

$$\neg(\neg \Phi \wedge \neg \Psi) \equiv \perp \quad \text{so } \Phi \vee \Psi \equiv \top$$

Es. 2

$$\neg A \wedge (\neg B \wedge ((C \wedge D) \rightarrow A)) \stackrel{CNF}{=} \neg A \wedge \neg B \wedge (\neg C \vee \neg D \vee A)$$

Es. 3

1) $[\neg A \vee \neg B] \longrightarrow [(A \wedge \neg B) \rightarrow (\neg B \vee C)] \equiv \perp$ ~~it is false when $A \equiv \top, B \equiv \perp, C \equiv \perp$~~

$$(A \wedge \neg B) \rightarrow (\neg B \vee C) \equiv \perp$$

$$A \wedge \neg B \equiv \top \quad A \equiv \top, B \equiv \perp$$

$$(\neg B \vee C) \not\equiv \perp \quad \text{so contradiction}$$

$$\neg A \vee \neg B \equiv \top \quad \text{so } \perp \vee \top \equiv \top$$

2) $[A \vee C] \longrightarrow [(A \rightarrow C) \rightarrow (C \rightarrow A)] \stackrel{?}{=} \perp$ it can be false when $A \equiv \perp, C \equiv \top$

$$(A \rightarrow C) \rightarrow (C \rightarrow A) \equiv \perp$$

$$A \rightarrow C \equiv \top \quad \text{so } \perp \rightarrow \top \equiv \top$$

$$C \rightarrow A \equiv \perp \quad \text{so } A \equiv \perp, C \equiv \top$$

$$A \vee C \equiv \top \quad \text{so } \perp \vee \top \equiv \top$$

Es. 4

$$1) (\neg A_k \wedge B_k) \vee (A_k \wedge \neg B_k)$$

$$2) [(A_k \wedge A_{k+1}) \rightarrow B_{k+2}] \wedge [(B_k \wedge B_{k+1}) \rightarrow A_{k+2}]$$

Es. 5

$$\text{ads}(x, y), \text{color}(x)$$

$$1) \forall x, y : \text{ads}(x, y) \rightarrow (\text{color}(x) \neq \text{color}(y))$$

$$2) \forall x, y, z : [\text{ads}(x, y) \wedge \text{ads}(y, z) \wedge \neg \text{ads}(x, z)] \rightarrow (\text{color}(x) = \text{color}(z))$$

Es. 6

$$\text{slws}(\text{nil}, \emptyset) :- !.$$

$$\text{slws}(t(-, \text{nil}, \text{nil}), \emptyset) :- !.$$

$$\text{slws}(t(-, t(-, \text{nil}, \text{nil}), \text{nil}), \lambda) :- !.$$

$$\text{slws}(t(-, \text{nil}, t(-, \text{nil}, \text{nil})), \lambda) :- !.$$

$$\text{slws}(t(-, L, R), N) :-$$

$$\text{slws}(L, N_1),$$

$$\text{slws}(R, N_2),$$

$$N \text{ is } N_1 \uparrow N_2.$$